

# Analysis: Cave Escape

## Cave Escape: Analysis

### Small dataset

Since there are no available potions in this dataset, we cannot increase our energy points and hence we are looking for a path with the smallest sum of traps which leads us to the exit. We can consider the cells of the grid as the vertices of our graph and moves between adjacent cells as the edges, with the weight of an edge being the absolute value of the destination cell. Running Dijkstra's algorithm on this graph gives us the path with the smallest sum of traps to reach the exit in  $O(NM * \log(NM))$  time.

### Large dataset

We can observe that whenever are potions that we can take without going through traps, we should always take them all before going through a trap. After that, we have two recursive choices :

- Go to the exit if it is reachable without going through a trap.
- Go through a trap cell which is reachable without going through any other trap. This choice can help us get more potions or help us reach the exit cell. After we choose to go through a trap cell, we should again take all potions that we can reach without going through an unvisited trap, and then repeat the choice above.

Let  $C_T$  be the count of traps in our test case. A naive recursive solution based on the above would be of the order of  $O(NM * C_T!)$ .

Observe that for a chosen subset of traps, all orderings of this subset have the same energy points (after going through all the traps), the same set of reachable unvisited traps and the same reachability of the exit cell. We can precompute these values by doing a graph search (BFS / DFS) on all  $2^{C_T}$  subsets. To find the order of visiting the traps, we can use dynamic programming along with bitmasks. For every mask of traps, we can choose to go to the next unvisited reachable trap if its cost is not greater than our current energy points, or go to the exit cell if it is reachable and take the choice that maximizes our answer.

The time complexity for precomputation is  $O(NM * 2^{C_T})$  and the time complexity of finding the ordering using dynamic programming is  $O(2^{C_T} * C_T)$ .

### Pseudocode

```
for every subset i in  $2^{C_T}$  subsets precompute :
     $E_i$  := Energy points left after visiting traps in subset i
     $R_i$  := Reachable unvisited traps after visiting traps in subset i
     $Exit_i$  := Denotes if exit cell is reachable from given subset of visited traps

def MaxEnergy(mask) {
    ret := -1

    // Go to exit if it is reachable
    if  $Exit_{mask} = \text{True}$  :
        ret :=  $E_{mask}$ 

    // Go to an unvisited reachable trap
    for i in  $R_{mask}$  :
        if  $TrapCost_i \leq E_{mask}$  :
            ret := max(ret, MaxEnergy(mask |  $2^i$ ))
}
```

```
    return ret  
}
```

# Analysis: Combining Classes

## Combining Classes: Analysis

### Small dataset

Let us define a function  $f(x)$  as the number of students scoring more than or equal to  $x$ . We can compute  $f(x)$  in  $O(N)$  time by iterating through all classes.

Therefore, we can use binary search to find the largest  $x$  that satisfies  $f(x) \geq K$ , which is the score of the  $K$ -th highest score. This solution runs in  $O(Q \times N \times \log(10^9))$  time.

### Large dataset

We can solve the Large dataset by compressing the scores first. That is, we map the existing values of  $L_i$  and  $R_i$  onto a range  $[1, D]$ , where  $D$  is the number of distinct values in  $L_i$  and  $R_i$ . After compressing the scores, we can assume that the students' scores are between 1 and  $2N$ .

For each possible score  $x$ , let  $\text{cnt}(x)$  be the number of students with score  $x$ . We can compute the values for all  $\text{cnt}(x)$  in  $O(N \log(N))$  using a range-update point-query data structure such as a segment tree. If we compute the suffix sum (similar to [prefix sum](#), but we cumulate the sum backwards) of  $\text{cnt}(x)$ , we will get the number of students scoring more than or equal to  $x$ , which is  $f(x)$ .

Therefore, just as in our solution for the Small dataset, we can use binary search to find the largest  $x$  that satisfies  $f(x) \geq K$ . However, note that the answer is not exactly  $x$ , since we compressed the scores earlier. The exact details of the implementation are left as an exercise.

This solution runs in  $O((N + Q) \log N)$  time.

# Analysis: Product Triplets

## Product Triplets: Analysis

### Small dataset

The Small dataset can be solved by complete search. We can check for each possible triplet  $(x, y, z)$  (with  $1 \leq x < y < z \leq N$ ) and check whether at least either one of  $A_x$ ,  $A_y$ , or  $A_z$  is the product of the other two. Each time we get a valid triplet, we increment our answer variable by one.

Since  $N \leq 200$  for this dataset, an  $O(N^3)$  time solution will run within the time limit.

### Large dataset

Let us ignore indices  $i$  where  $A_i = 0$  for the time being. In other words, let us assume that  $A_i > 0$  for all  $i$ .

We can observe that for any three positive integers  $a$ ,  $b$ , and  $c$ , if  $a \times b = c$ , then  $a \leq c$  and  $b \leq c$ . Therefore, if we sort  $A$  in nondecreasing order, for each triplet  $(x, y, z)$  (with  $1 \leq x < y < z \leq N$ ), we only need to check whether  $A_x \times A_y = A_z$ . However, doing this naively will still be an  $O(N^3)$  time solution.

We can optimize this solution by computing the number of  $z$  satisfying  $y < z$  and  $A_x \times A_y = A_z$  for each pair  $(x, y)$  in constant time. To do this, we can store the number of occurrences of each number in  $A$  in a hashmap.

If we design our nested loop such that we have  $y = \{N \dots 1\}$  as our outer loop (and  $x = \{y - 1 \dots 1\}$  as our inner loop), then we can update our hashmap for  $A_y$  just before we decrement the value of  $y$ . This is to make sure that our hashmap only contains the elements with indices larger than the current value of  $y$ . In other words, the algorithm looks something like:

```
sort(A)
ans = 0
occurrences = hashmap()
for y : {N .. 1}
    for x : {y - 1 .. 1}
        ans = ans + occurrences.count(A[x] * A[y])
    occurrences.update(A[y], occurrences.get(A[y]) + 1)
```

We should not forget that our algorithm so far ignores those indices  $i$  where  $A_i = 0$ . Suppose there are  $Z$  such indices (i.e., there are  $Z$  zeroes in  $A$ ). We can make  $C(Z, 3)$  triplets using three zeroes, and  $C(Z, 2) \times (N - Z)$  triplets using two zeroes and one non-zero.

Therefore, we get our final answer by summing the value of `ans` from our pseudocode, as well as  $C(Z, 3)$  and  $C(Z, 2) \times (N - Z)$ . This solution runs in  $O(N^2)$  time.